

Sorting and Patterns - Day 1

Sorted regions and two pointers

Priyanka's Platinum study plan | Monday, September 7, 2026 | 120 minutes | C11 and Python 3

Today's question: what promise lets us stop reconsidering part of an array? You will build bubble, selection, and insertion sort, then use sorted order to eliminate pair candidates and compact duplicates.

Prerequisites: array indexing, loops, functions, swapping, and C pointer parameters. All arrays contain integers. Sorts mutate their input into nondecreasing order. Pair and compaction inputs are already sorted. C callers provide valid array storage for n elements and valid output pointers; a null array is permitted only when n is zero. Assume 32-bit int and at least 64-bit long long for the numerical examples.

Your two-hour session

Minutes	Task	Deliverable
0-15	Trace the three sorts	Three invariant sentences
15-55	Lab A: implement the sorts in both languages	Six small functions
55-75	Count comparisons, swaps, shifts	Three input-family predictions
75-90	Derive pair sum	A justified pointer move
90-115	Labs B/C: pair sum and compaction, both languages	Correct contracts and edge tests
115-120	Exit ticket and corrections	One proof and one repaired bug

Attempt before opening the appendix. Begin in Python today, then translate the core function to C. Use the reference to unblock a stuck implementation within its time block; code-reading and correction count as guided practice. No extra homework is required by this schedule.

Discovery: three ways to earn a sorted region

Start with [5, 2, 4, 2, 1]. Sorting is more than preserving the multiset: we must establish order and terminate. For each algorithm, explain initialization, preservation, and what the invariant gives at termination.

Bubble sort. Scan adjacent pairs left to right, swapping only when the left is larger. After one pass the maximum reaches the final slot: [2, 4, 2, 1, 5]. After the second: [2, 2, 1, 4, 5]. The processed suffix holds the largest values in final positions; during a pass the largest value seen so far sits at the scan boundary. Reset the changed flag at the start of every pass. A pass with no swaps proves the remaining prefix is ordered, so the algorithm can stop.

Selection sort. Scan the entire unsorted suffix for its minimum, then swap it into the first unsorted position. The first pass gives [1, 2, 4, 2, 5]. Before iteration i , the prefix contains the i smallest values in sorted order, and each is no larger than any remaining value. Merely saying 'the prefix is sorted' misses the cross-boundary property.

Insertion sort. Before iteration i , $a[0:i]$ is sorted. Save $a[i]$ as key; shift larger prefix elements right until the hole reaches key's insertion position. The first two insertions produce [2, 5, 4, 2, 1], then [2, 4, 5, 2, 1]. While shifting, the key lives in a variable: the array alone temporarily contains a duplicate and is not a permutation of the original. Reinsert key to restore the multiset.

Lab A - Implement and discriminate

Write `bubble(a)`, `selection(a)`, `insertion(a)`, with C lengths passed explicitly. Do not call a library sort. For $n=0$ or 1, do no array access. In C, test $j>0$ before $a[j-1]$; unsigned `size_t` cannot represent -1. In Python, do not use slices to hide shifting or copying.

1. Finish all passes on [5,2,4,2,1]. Record the boundary after each pass.
2. Test [], [7], [1,2,3,4], [4,3,2,1], [2,2,2], and [-1,3,-1,0]. Check order AND element counts.
3. Predict key comparisons for $n=4$ sorted and reverse-sorted input. Distinguish comparisons from swaps and assignments.
4. Sort labelled records [2A,2B,1] by numeric key. Which algorithm reverses equal-key records? Define stability using these labels, not just identical integers.

Lab B - Pair sum without self-pairing

Given a sorted array and target, return any two distinct zero-based indices whose values sum to target. Python returns a tuple or None. C returns bool and writes two output indices only on success. Input is unchanged. Examples: [1,2,4,7], target 9 -> (1,3); [2,2], target 4 -> (0,1); [2], target 4 -> no solution.

Brute force tries $n(n-1)/2$ pairs. Instead place left at 0 and right at $n-1$. Invariant: every solution not yet ruled out lies entirely in [left,right]. If $a[\text{left}] + a[\text{right}]$ is too small, every pair using that left endpoint with a remaining partner is too small. If it is too large, every pair using the right endpoint is too large. Each move removes at least one candidate index. Stop when $\text{left} \geq \text{right}$.

Attempt: trace [-4,-1,0,3,5] with target 4, then target 20. Why must the loop use $\text{left} < \text{right}$? In C, cast an operand to long long BEFORE addition; assigning an overflowing int expression into long long is too late.

Lab C - A write pointer encodes the answer

Given a sorted array, keep one copy of each value in its first k slots and return k . Tail values are unspecified. Do not resize or allocate a second array. Example: [1,1,2,2,2,4] -> $k=3$ and prefix [1,2,4].

Use read to visit the input and write for the next output slot. Before each iteration, $a[0:\text{write}]$ holds the unique values from the processed input, in order; $\text{write} \leq \text{read}$, so writing cannot destroy future input. Compare the current value with the last written value, except when the output is empty. Trace the example with columns read, write, current value, and output prefix.

Tests: [], [5], [5,5,5], [-2,-2,0,1,1]. Tempting bug: unconditionally read $a[\text{write}-1]$. Explain why both languages mishandle the empty-prefix contract even though Python permits negative indexing.

Exit ticket

1. Why can insertion stop early on sorted input but selection still scans? Tomorrow you will make this quantitative.
2. Prove the pair-sum left move in one sentence using an inequality.
3. State compaction's logical length and physical length after completion.
4. Submit both language implementations, one trace per pattern, and your complexity explanation. Record the smallest failing input if anything remains unfinished.

Solution appendix

Open only after attempting the corresponding block. Reference functions follow after these checks.

Answers and cost model

For the supplied versions at $n=4$: bubble uses 3 key comparisons and 0 swaps on sorted input, 6 comparisons and 6 swaps on reverse input. Selection uses 6 comparisons on either input; this conditional-swap version makes 0 sorted-input swaps and 2 reverse-input swaps. Insertion uses 3 key comparisons and 0 shifts on sorted input, 6 comparisons and 6 shifts on reverse input. Loop-condition checks are not counted as key comparisons.

Bubble and insertion are stable because strict greater-than leaves equal-key order intact. Selection can change $[2A, 2B, 1]$ into $[1, 2B, 2A]$, so it is not stable.

All three use $O(1)$ auxiliary space. Bubble with early exit and insertion have $\Theta(n)$ best time and $\Theta(n^2)$ worst time. Selection is $\Theta(n^2)$ on every input. For uniformly random permutations of distinct keys their average time is $\Theta(n^2)$. For $n < 2$, treat work as constant.

Pair sum and compaction take $O(n)$ time and $O(1)$ workspace. For target 4 on $[-4, -1, 0, 3, 5]$, discard -4 after sum 1, then return indices (1,4). Target 20 repeatedly advances left and returns failure. Every iteration shortens the candidate interval. Compaction returns a logical length of 3 for $[1, 1, 2, 2, 2, 4]$, while storage still has six slots.

Progressive hints

Sorts: first identify the final region; then identify the boundary index. Pair sum: compare the largest available partner for the left value. Compaction: treat write as the length of a valid answer, not as a second reader. If stuck, dry-run only two equal elements before returning to longer inputs.

Reference implementation contract

The following functions are also supplied as downloadable .py and .c files in Aleph. They contain no input/output harness so you can reuse them in later days. Compile C with `-std=c11 -Wall -Wextra -Werror -c`. In Python, call a sort then inspect the mutated list; these sorting functions return None.

Python 3 reference functions

PY

```
def bubble(a):
    for end in range(len(a) - 1, 0, -1):
        changed = False
        for j in range(end):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                changed = True
        if not changed:
            break
```

PY

```
def selection(a):
    for i in range(len(a) - 1):
        smallest = i
        for j in range(i + 1, len(a)):
            if a[j] < a[smallest]:
                smallest = j
        if smallest != i:
            a[i], a[smallest] = a[smallest], a[i]
```

PY

```
def insertion(a):
    for i in range(1, len(a)):
        key = a[i]
        j = i
        while j > 0 and a[j - 1] > key:
            a[j] = a[j - 1]
            j -= 1
        a[j] = key
```

PY

```
def pair_sum(a, target):
    left, right = 0, len(a) - 1
    while left < right:
        total = a[left] + a[right]
        if total == target:
            return left, right
        if total < target:
            left += 1
        else:
            right -= 1
    return None
```

PY

```
def unique_prefix(a):
    write = 0
    for read in range(len(a)):
        if write == 0 or a[read] != a[write - 1]:
            a[write] = a[read]
            write += 1
    return write
```

C11 reference functions

C

```
#include <stdbool.h>
#include <stddef.h>
```

C

```
void bubble(int *a, size_t n) {
    for (size_t end = n; end > 1; --end) {
        bool changed = false;
        for (size_t j = 0; j + 1 < end; ++j) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                changed = true;
            }
        }
        if (!changed) break;
    }
}
```

C

```
void selection(int *a, size_t n) {
    for (size_t i = 0; i + 1 < n; ++i) {
        size_t smallest = i;
        for (size_t j = i + 1; j < n; ++j)
            if (a[j] < a[smallest]) smallest = j;
        if (smallest != i) {
            int t = a[i]; a[i] = a[smallest]; a[smallest] = t;
        }
    }
}
```

C

```
void insertion(int *a, size_t n) {
    for (size_t i = 1; i < n; ++i) {
        int key = a[i];
        size_t j = i;
        while (j > 0 && a[j - 1] > key) {
            a[j] = a[j - 1];
            --j;
        }
        a[j] = key;
    }
}
```

C

```
bool pair_sum(const int *a, size_t n, long long target,
              size_t *out_left, size_t *out_right) {
    if (n < 2) return false;
    size_t left = 0, right = n - 1;
    while (left < right) {
        long long total = (long long)a[left] + a[right];
        if (total == target) {
            *out_left = left; *out_right = right;
            return true;
        }
        if (total < target) ++left;
        else --right;
    }
    return false;
}
```

C

```
size_t unique_prefix(int *a, size_t n) {
    size_t write = 0;
    for (size_t read = 0; read < n; ++read) {
        if (write == 0 || a[read] != a[write - 1])
            a[write++] = a[read];
    }
    return write;
}
```